

Managing Variables and Secrets: An Overview

In today's development environments, proper management of variables and secrets is critical for maintaining security and operational efficiency.

- **Variables:** Store configuration data like environment settings and build parameters
- **Secrets:** Handle sensitive information such as passwords, API tokens, and certificates
- **Protection:** Require special safeguards to prevent unauthorized access

When secrets are exposed, organizations face serious consequences:

- Data breaches and system compromises
- Unauthorized access to critical infrastructure
- Compliance violations and regulatory penalties
- Loss of customer trust and reputation damage

Implementing automated secret management provides significant benefits:

- Reduces human error in handling sensitive data
- Strengthens overall security posture
- Ensures consistent protection across environments
- Enables audit trails and compliance reporting



Types of Variables in CI/CD

Variables in continuous integration and deployment pipelines serve as essential key-value pairs that control how your automation works. They come in two primary categories:

1

Standard Variables

Configuration values that help customize pipeline behavior:

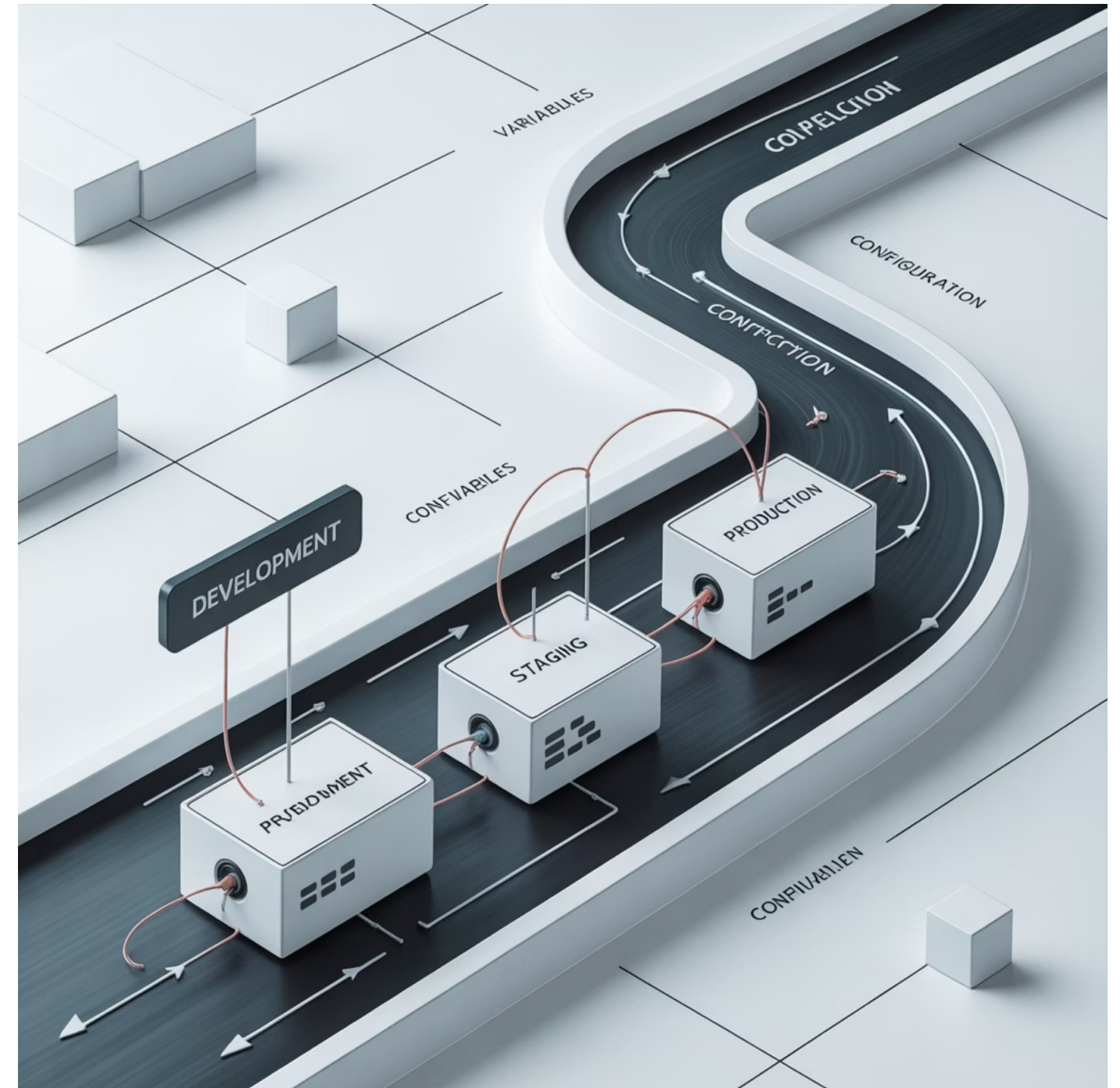
- Environment settings and paths
- Build parameters and flags
- URLs and endpoints
- Non-sensitive configuration data

2

Secret Variables

Sensitive information requiring special protection:

- Passwords and authentication tokens
- API keys and access credentials
- Encryption keys and certificates
- Database connection strings



Variables enable consistent deployment across environments:

Protected Variables



Key Features:

- Only accessible when pipelines run on protected branches or tags
- Ideal for production credentials that should never be exposed to feature branches
- Prevents accidental exposure in non-production environments
- Enforces separation between development and production access

Protected variables create a secure boundary between production environments and development work, reducing the risk of credential leakage.

Protected variables provide an additional layer of security by restricting access to

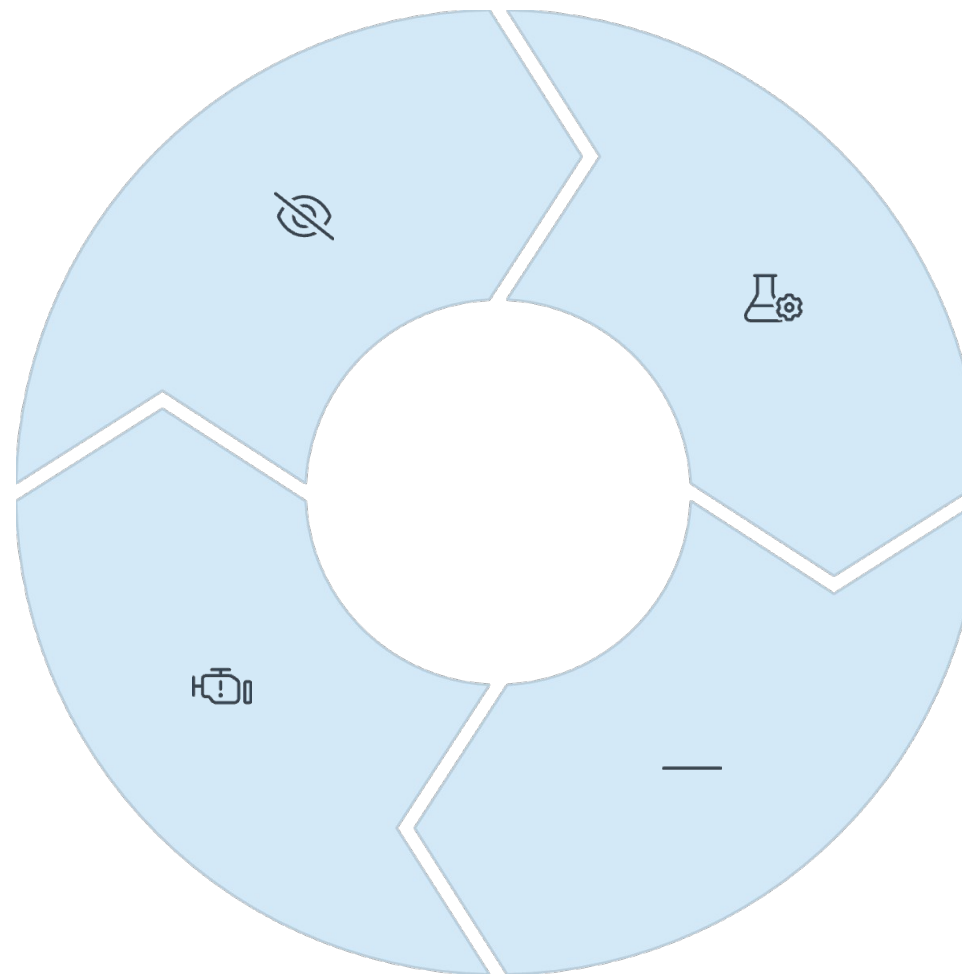
Masked Variables

Hide Values in Logs

Displays [MASKED] instead of actual values in pipeline job logs, preventing casual observation of sensitive data.

Limitations

Not completely foolproof - values can still be exposed if debugging features are enabled or output is manipulated.



Configuration Options

Can be set at project, group, or admin level depending on your organizational needs and security policies.

Technical Requirements

Must be single-line values with at least 8 characters, using a limited character set to enable reliable masking.

Limitations of Masked Variables

While masked variables provide a layer of protection, they have important limitations that security-conscious teams must understand:

Log Obfuscation Only

Masking only replaces the value in logs, but the actual value remains accessible to the internal process and any scripts it runs.

Potential Exposure

Malicious code could intentionally print secret values in modified formats that bypass masking patterns (e.g., inserting spaces, encoding).

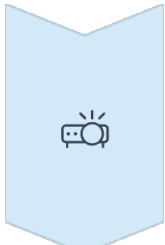
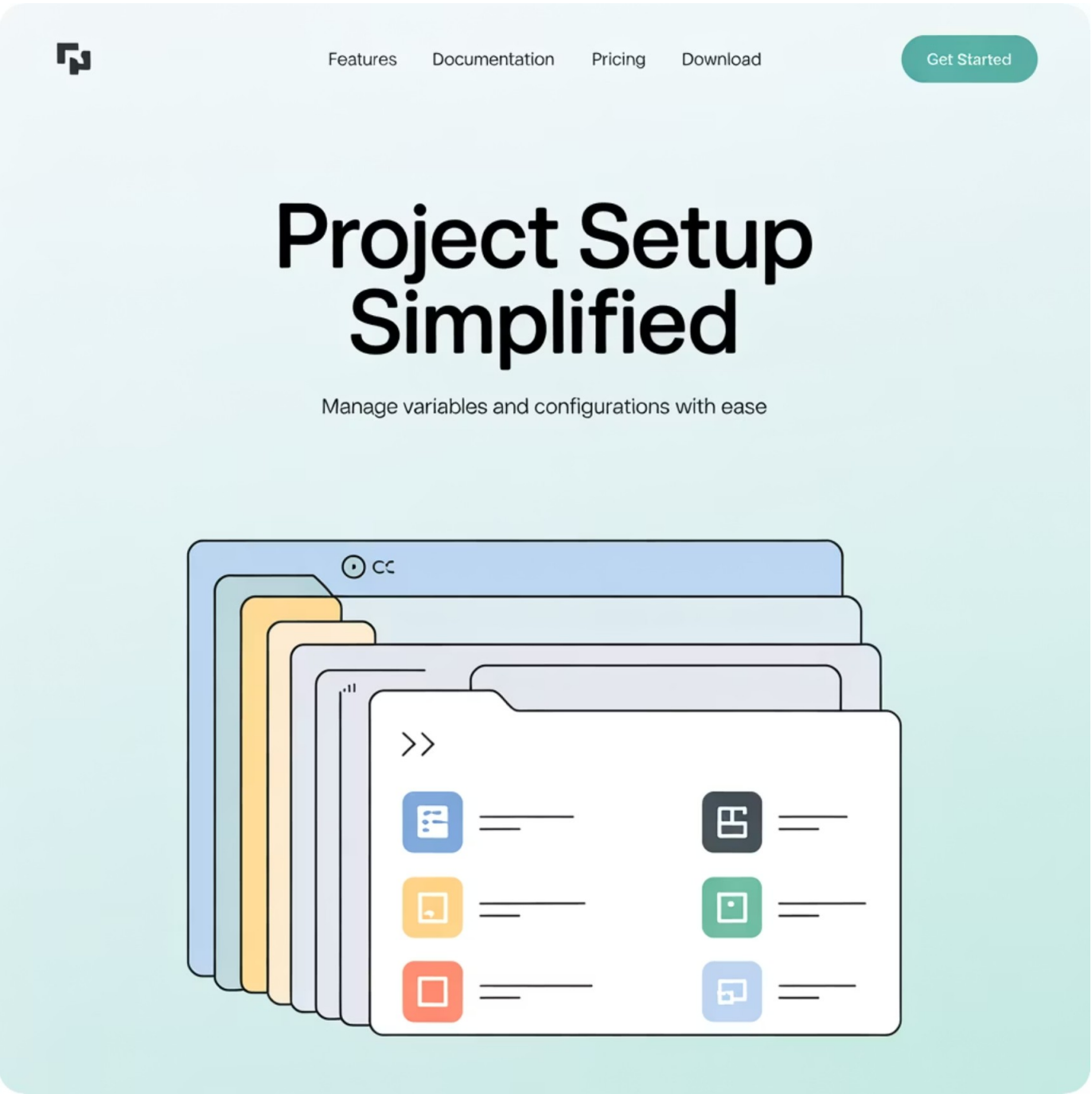
Higher Security Needs

For truly sensitive secrets, external vault services like HashiCorp Vault or cloud provider secret managers provide stronger protection.



Understanding these limitations is crucial for implementing appropriate security

Project-Level Variables



Project Scope

Only accessible within the specific project where they're defined, keeping configuration isolated.



Override Capability

Can override group-level variables with the same name, allowing customization for specific projects.



Environment Control

Perfect for project-specific settings like deployment targets, feature flags, and application-specific secrets.

Project-level variables provide configuration specific to a single repository or

Group-Level Variables



Organization

Top-level groups for broad organizational settings like compliance and security standards.



Division/Team

Mid-level groups for team-specific configurations shared across related projects.



Project

Inherited variables available to all pipelines in the project unless overridden.

Group-level variables enable centralized management of configuration across multiple projects. Update a value once at the group level, and all child projects automatically use the new value—perfect for shared infrastructure credentials, common endpoints, or organization-wide settings.

Best Practices for Managing Variables and Secrets

Automate Secret Management

Use dedicated tools like HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault to handle secret rotation, auditing, and access control.

Implement Regular Rotation

Frequently update secrets (every 30-90 days) to minimize the risk window if credentials are compromised.

Apply Least Privilege

Restrict variable access to only the branches, environments, and team members who absolutely need it.

Combine Protection Methods

Use both masking and protection together for sensitive values, understanding the limitations of each approach.



Remember that proper secret management is a continual process, not a one-time setup.